

Implement stateless JWT with refresh tokens and rotation

Implementing stateless JWT authentication with refresh tokens and rotation in ASP.NET Core (with .NET 8) involves issuing short-lived access tokens together with longer-lived refresh tokens, and on each refresh, rotating (revoking old and issuing a new) refresh token.

Key Concepts:

- Access tokens are short-lived JWTs used to authenticate API requests.
- Refresh tokens are used to request new access tokens after expiration.
- Refresh token rotation invalidates the old refresh token immediately upon use, mitigating risks from token theft.
- Refresh tokens are stored server-side or tracked to enable revocation.

Example Implementation Overview

1. Models: Create RefreshToken entity to track tokens, expiration, revocation.
2. Token Service: Generate JWT tokens and refresh tokens. On refresh, revoke old refresh token and generate new ones.
3. Authentication Controller: Endpoint for login returning access + refresh tokens, refresh endpoint for token rotation.
4. Middleware: Use JWT bearer for access tokens.

Simplified Code Snippets

RefreshToken Entity (EF Core) public

```
class RefreshToken
{
    public int Id { get; set; }

    public string Token { get; set; } = null!;

    public string JwtId { get; set; } = null!; // Links refresh token to access JWT

    public DateTime Expires { get; set; }

    public bool IsRevoked { get; set; } = false;

    public DateTime? RevokedAt { get; set; }
```

```
public int UserId { get; set; }

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
public string CreatedByIp { get; set; } = null!;
}

```

TokenService Example (Generate Tokens & Rotate)

```
public
class TokenService
{
    private IConfiguration _config;

    public TokenService(IConfiguration config) { _config = config; }

    public string GenerateAccessToken(User user, out string jwtId)
    {
        jwtId = Guid.NewGuid().ToString();

        var claims = new List<Claim>
        {
            new Claim(JwtRegisteredClaimNames.Jti, jwtId),
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString())

            // Add other claims
        };

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience:
            _config["Jwt:Audience"],
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(15),
            signingCredentials: creds);
    }
}

```

```
return new JwtSecurityTokenHandler().WriteToken(token);  
}
```

```
public RefreshToken GenerateRefreshToken(string ipAddress, string jwtId, int userId)  
{  
    return new RefreshToken  
    {  
        Token = Convert.ToBase64String(RandomNumberGenerator.GetBytes(64)),  
        JwtId = jwtId,  
        Expires = DateTime.UtcNow.AddDays(7),  
        UserId = userId,  
        CreatedByIp = ipAddress  
    };  
}
```

Refresh Endpoint Logic [HttpPost("refresh")]

```
public async Task<ActionResult> Refresh(TokenRequest request)  
{  
    var principal = GetPrincipalFromExpiredToken(request.AccessToken);    var  
    userId = int.Parse(principal.FindFirstValue(ClaimTypes.NameIdentifier));  
  
    var savedRefreshToken = await _dbContext.RefreshTokens  
        .SingleOrDefaultAsync(rt => rt.Token == request.RefreshToken && !rt.IsRevoked);  
  
    if (savedRefreshToken == null || savedRefreshToken.Expires <= DateTime.UtcNow)    return  
    Unauthorized("Invalid refresh token");
```

```
savedRefreshToken.IsRevoked = true; // revoke old token    savedRefreshToken.RevokedAt =
DateTime.UtcNow;

var jwtId = Guid.NewGuid().ToString();
var newAccessToken = _tokenService.GenerateAccessToken(user, out jwtId);
var newRefreshToken = _tokenService.GenerateRefreshToken(request.IpAddress, jwtId, userId);

_dbContext.RefreshTokens.Add(newRefreshToken);    await
_dbContext.SaveChangesAsync();

return Ok(new
{
    AccessToken = newAccessToken,
    RefreshToken = newRefreshToken.Token
});
}
```

JwtBearer & Authentication Setup

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)    .AddJwtBearer(options
=>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
```

```
ValidAudience = builder.Configuration["Jwt:Audience"],  
IssuerSigningKey = new SymmetricSecurityKey(  
    Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))  
};  
});
```

```
app.UseAuthentication(); app.UseAuthorization();
```

Best Practices

- Make access tokens short-lived (e.g., 5-15 minutes).
- Use secure HttpOnly cookies to store refresh tokens on the client if possible.
- Implement token revocation and reuse detection to stop token theft abuse.
- Invalidate refresh tokens immediately upon use (rotation).
- Log token usage and revocation events for audits.

This approach keeps the system stateless for access tokens while securely managing refresh tokens server-side to achieve scalability and security in ASP.NET Core backends. If you want, a full sample project can be provided for .NET 8 minimal API or controller API style.

Show a secure refresh-token rotation flow for SPAs using HTTP-only cookies

A secure refresh-token rotation flow for Single Page Applications (SPAs) using HTTP-only cookies involves storing the refresh token in an HTTP-only, secure cookie, which is inaccessible to JavaScript. This protects against XSS attacks while allowing the browser to send the refresh token automatically with each refresh request. The access token is usually stored in memory or a non-persistent storage on the client.

Secure Refresh Token Rotation Flow for SPAs

1. Login

- Server issues a short-lived access token (returned in response body or header).
- Server issues a refresh token stored in HttpOnly, Secure, SameSite=Strict cookie.
- Client stores the access token in memory.

2. Accessing Resources

- Client sends access token in Authorization header Bearer <access_token>.
- Server validates access token.
- If access token is expired, client triggers refresh.

3. Refresh Token Request

- Client calls refresh endpoint without sending tokens manually; the HttpOnly cookie with the refresh token is sent automatically.
- Server validates refresh token and rotates it by invalidating the old refresh token and issuing a new refresh token cookie and new access token.
- Server sets a new HttpOnly refresh token cookie in response.
- Client updates its access token in memory.

4. Logout / Token Revocation

- Server deletes refresh token (revokes).
- Server instructs cookie removal by sending expired cookie.
- Client clears access token from memory.

Example ASP.NET Core Implementation Highlights

```
// Configure cookie options
```

```
var cookieOptions = new CookieOptions
```

```
{
```

```
    HttpOnly = true,
```

```
    Secure = true,
```

```
    SameSite = SameSiteMode.Strict,
```

```
    Expires = DateTime.UtcNow.AddDays(7),
```

```
    Path = "/"
```

```
};
```

```
// On login or refresh token creation, set refresh token cookie
```

```
Response.Cookies.Append("refreshToken", refreshToken.Token, cookieOptions);
```

```
// Refresh token endpoint
```

```
[HttpPost("refresh")]
```

```
public async Task<IActionResult> Refresh()
```

```
{
```

```
    var refreshToken = Request.Cookies["refreshToken"];
```

```
    if (string.IsNullOrEmpty(refreshToken))
```

```
        return Unauthorized();
```

```
    var savedToken = await _dbContext.RefreshTokens
```

```
        .SingleOrDefaultAsync(rt => rt.Token == refreshToken && !rt.IsRevoked && rt.Expires >
        DateTime.UtcNow);
```

```
    if (savedToken == null)
```

```
        return Unauthorized();
```

```
// Rotate refresh token: revoke old, create new token entry

savedToken.IsRevoked = true;

savedToken.RevokedAt = DateTime.UtcNow;


var newRefreshToken =
GenerateRefreshToken(Request.HttpContext.Connection.RemoteIpAddress?.ToString(),
savedToken.JwtId, savedToken.UserId);

_dbContext.RefreshTokens.Add(newRefreshToken);

await _dbContext.SaveChangesAsync();


// Set new HttpOnly cookie for new refresh token

Response.Cookies.Append("refreshToken", newRefreshToken.Token, cookieOptions);


// Generate new access token

var user = await _userService.GetUserByIdAsync(savedToken.UserId);

var jwtId = string.Empty;

var accessToken = _tokenService.GenerateAccessToken(user, out jwtId);

return Ok(new { AccessToken = accessToken });
}
```

Benefits of This Flow

- Refresh token never exposed to JavaScript, mitigating XSS risk.
- Cookie attributes Secure and SameSite=Strict enhance protection against CSRF.
- Refresh tokens rotated on every refresh request to minimize risk from token theft.
- Access token is short-lived and stored in memory (not persisted), limiting exposure.

Client-side Tips for SPA

- Store access token in memory, not localStorage or sessionStorage.

API Design and backend Developer Concepts

- On 401 responses, automatically call refresh endpoint to get a new access token.
- Clear all tokens on logout.

This secure refresh token rotation pattern with HTTP-only cookies is the best practice for SPAs interacting with ASP.NET Core backends for highly secure, scalable authentication.

API Versioning:

ASP.NET Core provides multiple methods and tools to implement API versioning, usually through the official API versioning library and configurable strategies. There are four primary approaches supported by built-in tools and widely accepted best practices.

Main Versioning Methods

- Query String Parameter

The API version is passed as a query string, e.g., `api/products?api-version=1.0`.

```
// In Startup.cs or Program.cs - configure services
```

```
services.AddApiVersioning(options =>
```

```
{
```

```
    options.ApiVersionReader = new QueryStringApiVersionReader("api-version");
```

```
    options.AssumeDefaultVersionWhenUnspecified = true;
```

```
    options.DefaultApiVersion = new ApiVersion(1, 0);
```

```
});
```

API call example:

GET `/api/products?api-version=1.0`

- URL Segment Versioning

The version is part of the route, such as `/api/v1/products`.

```
// Configure service
```

```
services.AddApiVersioning(options =>
```

```
{
```

```
    options.ApiVersionReader = new UrlSegmentApiVersionReader();
```

```
    options.AssumeDefaultVersionWhenUnspecified = true;
```

```
    options.DefaultApiVersion = new ApiVersion(1, 0);
```

```
});
```

```
// Controller route example
```

```
[ApiVersion("1.0")]
```

```
[Route("api/v{version:apiVersion}/products")]  
[ApiController]  
public class ProductsV1Controller : ControllerBase  
{  
    // Actions here  
}
```

API call example:

GET /api/v1/products

- Header Versioning

The version is supplied as a custom HTTP header, for example, X-API-Version: 1.0 or x-version: 1.0.

```
services.AddApiVersioning(options =>  
{  
    options.ApiVersionReader = new HeaderApiVersionReader("X-API-Version");  
    options.AssumeDefaultVersionWhenUnspecified = true;  
    options.DefaultApiVersion = new ApiVersion(1, 0);  
});
```

API call example:

GET /api/products

X-API-Version: 1.0

- Media Type Versioning

The version is provided in the Accept header as a media type parameter: application/json; ver=1.0.

```
services.AddApiVersioning(options =>  
{  
    options.ApiVersionReader = new MediaTypeApiVersionReader("ver");  
    options.AssumeDefaultVersionWhenUnspecified = true;  
    options.DefaultApiVersion = new ApiVersion(1, 0);  
});
```

API call example with header:

Accept: application/json; ver=1.0

GET /api/products

ASP.NET Core Versioning Tools

- **Microsoft.AspNetCore.Mvc.Versioning**
The official NuGet package for ASP.NET Core API versioning, supporting all methods listed above.
- **Asp.Versioning.Mvc** and **Asp.Versioning.Http**
Additional packages for more scenarios (minimal APIs, controller-based APIs, API explorer support).

Developers can also implement custom `ApiVersionReader` strategies for unique requirements, further expanding the available methods.

Summary Table

Versioning Method	Example Usage	Supported by Official Package
Query String Parameter	?api-version=1.0	Yes
URL Segment	/api/v1/products	Yes
Header	X-API-Version: 1.0	Yes
Media Type	application/json; ver=1.0	Yes
Custom Reader	Custom logic (bespoke)	Yes

In summary, ASP.NET Core supports four main methods for API versioning, all configurable via the API versioning libraries and customizable through extension points.

Company	Versioning Method	URL Example	Pros
Twitter	URL Path	/1.1/endpoint and /2/endpoint	Simple, cache-friendly, visible
GitHub	Header	Accept: application/vnd.github.v3+json	Clean URLs, flexible
Facebook	Query Parameter / URL	/v8.0/me?access_token=...	Easy testing, straightforward

Security or authentication Scaling for backend Developer in Asp.Net core

To securely and scalably implement authentication in ASP.NET Core for backend development, follow these patterns and tools designed for high-traffic, distributed systems.

Scalable Authentication Methods

- JWT (JSON Web Tokens):
 - Enables stateless authentication, allowing APIs to scale horizontally without server-side session storage.
 - Tokens are validated on every request using middleware, with user details encoded in the token—no central session required.
- API Keys:
 - Useful for machine-to-machine scenarios, providing simple, scalable authentication for non-user-specific requests.
- OAuth 2.0 & OpenID Connect:
 - Delegate authentication to cloud providers or custom identity servers, facilitating multi-tenant and federated authentication needs.
 - Supports external logins, complex token flows, and is highly scalable when paired with distributed identity providers.

Scalability Considerations

- Distributed State:
 - Avoid server-side sessions; use stateless tokens (JWT) to keep authentication scalable across multiple instances.
- Centralized Data Protection:
 - For features (forms, antiforgery) relying on ASP.NET Core's data protection, configure shared key storage (e.g., Azure Key Vault, Redis) so all servers can decrypt tokens and cookies consistently.
- Caching User Profiles:
 - Reduce database hits by caching user claims and profiles after token validation (Redis, in-memory cache).
- Multitenancy:
 - Use distributed identity systems, custom user stores per tenant, or identity providers like Azure AD to support multi-tenant scaling.

API Design and backend Developer Concepts

- Rate Limiting and HTTPS:
 - Always enable HTTPS, apply rate limiting to throttle abuse, and ensure input validation on every endpoint.

Best Practices

- Enforce short JWT lifetimes and use refresh tokens.
- Always verify token signature, expiration, issuer, and audience.
- For role and claims-based authorization, use attributes and policies—scalable and maintainable.
- In distributed APIs, sync data protection keys for cookies and antiforgery to allow scaling across servers/cloud.

Popular API Design Tools

Tool	Description
Swagger/OpenAPI	Industry-standard specification for REST APIs with tools for design, documentation, testing
Postman	UI tool for designing, testing, documenting APIs collaboratively
Apidog	Visual API builder with collaboration and automated documentation
Stoplight	API design platform supporting OpenAPI specs and style guides
Astera	No-code API builder with drag-drop, transformations, and integration
Insomnia	API client and design environment supporting REST, GraphQL
Kong	API gateway with design and management features

API Design in ASP.NET core web API in .NET 8

To start API design in ASP.NET Core Web API with .NET 8, the approach includes setting up the development environment, creating a new ASP.NET Core Web API project, defining models, creating controllers for handling requests, and configuring services including Entity Framework Core if a database is used. Here's a step-by-step example:

Step 1: Set Up Development Environment

- Install .NET 8 SDK.
- Use Visual Studio or Visual Studio Code.

Step 2: Create a New ASP.NET Core Web API Project

- Use the command line or Visual Studio project creation wizard.
- Ensure the project targets .NET 8.

Step 3: Understand Project Structure

- Controllers folder: API controllers.
- Program.cs: Main entry point, middleware setup.
- appsettings.json: Configuration settings.
- LaunchSettings.json: Environment profiles.

Step 4: Define Data Models

Create a model to represent your data.

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Step 5: Create Database Context (Optional, with EF Core)

Set up EF Core for database access.

Example:

```
public class MyDbContext : DbContext
{
    public MyDbContext(DbContextOptions<MyDbContext> options) : base(options) {}
    public DbSet<Product> Products { get; set; }
}
```

Step 6: Configure Services in Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddDbContext<MyDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

```
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
```

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

```
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Step 7: Create a Controller

Example controller for CRUD operations on Product:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly MyDbContext _context;

    public ProductsController(MyDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public IActionResult GetProducts()
    {
        var products = _context.Products.ToList();
        return Ok(products);
    }

    [HttpGet("{id}")]
    public IActionResult GetProduct(int id)
    {
        var product = _context.Products.Find(id);
        if (product == null) return NotFound();
        return Ok(product);
    }
}
```

```
}

[HttpPost]
public IActionResult CreateProduct(Product product)
{
    _context.Products.Add(product);
    _context.SaveChanges();
    return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
}

[HttpPut("{id}")]
public IActionResult UpdateProduct(int id, Product product)
{
    if (id != product.Id) return BadRequest();
    var existing = _context.Products.Find(id);
    if (existing == null) return NotFound();

    existing.Name = product.Name;
    existing.Price = product.Price;
    _context.SaveChanges();
    return NoContent();
}

[HttpDelete("{id}")]
public IActionResult DeleteProduct(int id)
{
    var product = _context.Products.Find(id);
    if (product == null) return NotFound();

    _context.Products.Remove(product);
    _context.SaveChanges();
    return NoContent();
}
}
```

This basic example covers setting up a RESTful API with CRUD operations using .NET 8 and ASP.NET Core Web API.

Minimal API Design in ASP.NET core web API in .NET 8

Minimal API design in ASP.NET Core with .NET 8 is a streamlined approach that reduces boilerplate and simplifies building API endpoints directly in the Program.cs file without controllers. Here's a step-by-step example and explanation for creating a minimal API with basic CRUD operations:

Step 1: Create a .NET 8 Minimal API Project

This creates a project with Program.cs as the main file.

Step 2: Define the Model

For example, a simple CompanyItem model:

```
public class CompanyItem
{
    public int Id { get; set; }
    public string Technology { get; set; }
    public string Vacancy { get; set; }
}
```

Step 3: In-Memory Data Store

Using static data for simplicity:

```
public static class CompanyData
{
    public static List<CompanyItem> Items { get; } = new List<CompanyItem>
    {
        new CompanyItem { Id = 1, Technology = "Laravel", Vacancy = "No" },
        new CompanyItem { Id = 2, Technology = "Android", Vacancy = "Yes" },
        new CompanyItem { Id = 3, Technology = "Dot Net", Vacancy = "Yes" }
    };
}
```

Step 4: Configure the Minimal API Endpoints in Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add Swagger/OpenAPI support for API documentation.
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
```

```
app.UseSwagger();
app.UseSwaggerUI();

// Middleware to set JSON response content type.
app.Use(async (context, next) =>
{
    context.Response.Headers.Add("Content-Type", "application/json");
    await next();
});

// GET all items
app.MapGet("/Companytodos", () => CompanyData.Items);

// GET item by ID
app.MapGet("/Companytodos/{id}", (int id) =>
{
    var company = CompanyData.Items.FirstOrDefault(t => t.Id == id);
    return company is not null ? Results.Ok(company) : Results.NotFound();
});

// POST create new item
app.MapPost("/Companytodos", async (HttpContext context) =>
{
    var company = await context.Request.ReadFromJsonAsync<CompanyItem>();
    if (company is null) return Results.BadRequest("Invalid item");

    company.Id = CompanyData.Items.Count > 0 ? CompanyData.Items.Max(t => t.Id) + 1 : 1;
    CompanyData.Items.Add(company);
    return Results.Created($" /Companytodos/{company.Id}", company);
});

// PUT update an item
app.MapPut("/Companytodos/{id}", async (HttpContext context, int id) =>
{
    var company = CompanyData.Items.FirstOrDefault(t => t.Id == id);
    if (company is null) return Results.NotFound();

    var updatedCompany = await context.Request.ReadFromJsonAsync<CompanyItem>();
    if (updatedCompany is null) return Results.BadRequest("Invalid item");

    company.Technology = updatedCompany.Technology;
    company.Vacancy = updatedCompany.Vacancy;
```

```
        return Results.Ok(company);
    });

// DELETE an item
app.MapDelete("/Companytodos/{id}", (int id) =>
{
    var company = CompanyData.Items.FirstOrDefault(t => t.Id == id);
    if (company is null) return Results.NotFound();

    CompanyData.Items.Remove(company);
    return Results.Ok();
});

app.Run();
```

Advantages of Minimal APIs in .NET 8

- Simplifies API development with minimal code and boilerplate.
- Improved performance via less abstraction.
- Good for microservices, prototyping, and lightweight APIs.
- Easy to learn and extend.
- Supports modern features like async/await and dependency injection.

This example runs a complete minimal API with endpoints for Create, Read, Update, and Delete operations on a simple in-memory collection.

Would you like a detailed explanation on integrating Entity Framework Core or handling authentication in minimal APIs?

This is based on recent examples and best practices for minimal API design in ASP.NET Core 8.

API design with GraphQL

Designing a GraphQL API in ASP.NET Core with .NET 8 using the Hot Chocolate library, which is a popular GraphQL server implementation for .NET.

Step 1: Create a New ASP.NET Core Web API Project

Step 2: Add Required NuGet Packages

Install Hot Chocolate packages for GraphQL support:

```
dotnet add package HotChocolate.AspNetCore
dotnet add package HotChocolate.Data
dotnet add package HotChocolate.Types
```

Step 3: Define Your Data Model

Create a simple data model class, e.g., Product:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Step 4: Define Query Type

Create a class that serves as the Query entry point for GraphQL. This will expose the queryable data.

```
public class Query
{
    private static readonly List<Product> products = new()
    {
        new Product { Id = 1, Name = "Laptop", Price = 1200 },
        new Product { Id = 2, Name = "Phone", Price = 800 },
        new Product { Id = 3, Name = "Tablet", Price = 450 }
    };

    public IEnumerable<Product> GetProducts() => products;

    public Product GetProduct(int id) => products.FirstOrDefault(p => p.Id == id);
}
```

Step 5: Configure GraphQL Services in Program.cs

Modify Program.cs to add GraphQL services and map the GraphQL endpoint:

```
var builder = WebApplication.CreateBuilder(args);

// Add GraphQL server and register Query type
builder.Services
    .AddGraphQLServer()
    .AddQueryType<Query>();

var app = builder.Build();

app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapGraphQL(); // Maps /graphql endpoint by default
});

app.Run();
```

Step 6: Run and Test the API

Run the application:

Use the GraphQL playground available at <http://localhost:5000/graphql> (or the port your app runs on) to test queries like:

```
query {
  products {
    id
    name
    price
  }
  product(id: 2) {
    id
    name
  }
}
```

Optional: Adding Mutation to Modify Data

Add a mutation class to enable creating, updating, or deleting data.

```
public class Mutation
{
    private static readonly List<Product> products = Query.products;

    public Product AddProduct(string name, decimal price)
    {
        var newProduct = new Product
        {
            Id = products.Max(p => p.Id) + 1,
            Name = name,
            Price = price
        };
        products.Add(newProduct);
        return newProduct;
    }
}
```

Register mutation type in Program.cs:

```
builder.Services
    .AddGraphQLServer()
    .AddQueryType<Query>()
    .AddMutationType<Mutation>();
```

Then, you can run mutations such as:

```
mutation {
  addProduct(name: "Smartwatch", price: 299) {
    id
    name
    price
  }
}
```

This step-by-step guide sets up a basic GraphQL API on .NET 8 with ASP.NET Core using Hot Chocolate. You get the flexibility of GraphQL queries and mutations in a strongly typed environment.

Web Socket

WebSocket is a protocol that provides full-duplex communication channels over a single TCP connection, allowing real-time, two-way interactive communication between a client and a server. Unlike traditional HTTP requests, WebSockets enable a persistent connection, making it ideal for applications requiring instant data updates, such as chat applications, live dashboards, or gaming.

Purpose of WebSocket

- Facilitates real-time communication with low latency.
- Reduces overhead compared to repeated HTTP requests.
- Maintains a persistent connection, enabling instant data transfer between client and server.
- Used in scenarios like chat apps, live notifications, real-time collaboration, and gaming.

Why Use WebSocket in ASP.NET Core Web API

- To build applications that need real-time features.
- To implement bidirectional communication efficiently.
- To improve performance and reduce server load by avoiding repeated HTTP requests.

How to Implement WebSocket in ASP.NET Core Web API: Step-by-Step

Step 1: Create ASP.NET Core Project

Create a new ASP.NET Core Web API project:

Step 2: Add WebSocket Middleware in Program.cs

Configure middleware to accept WebSocket requests:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseWebSockets();

app.Map("/ws", async context =>
{
    if (context.WebSockets.IsWebSocketRequest)
    {
        using var webSocket = await context.WebSockets.AcceptWebSocketAsync();
        await HandleWebSocketAsync(webSocket);
    }
    else
    {
        context.Response.StatusCode = 400;
    }
});

app.Run();
```

```
async Task HandleWebSocketAsync(WebSocket webSocket)
{
    var buffer = new byte[1024 * 4];
    WebSocketReceiveResult result;

    // Keep listening to messages
    while (webSocket.State == WebSocketState.Open)
    {
        result = await webSocket.ReceiveAsync(new ArraySegment<byte>(buffer),
        CancellationToken.None);

        if (result.MessageType == WebSocketMessageType.Text)
        {
            var message = Encoding.UTF8.GetString(buffer, 0, result.Count);
            // Process message (echo response here)
            var responseMsg = $"Server echo: {message}";
            var responseBytes = Encoding.UTF8.GetBytes(responseMsg);
            await webSocket.SendAsync(new ArraySegment<byte>(responseBytes),
            WebSocketMessageType.Text, true, CancellationToken.None);
        }
        else if (result.MessageType == WebSocketMessageType.Close)
        {
            await webSocket.CloseAsync(WebSocketCloseStatus.NormalClosure, "Closing",
            CancellationToken.None);
        }
    }
}
```

Step 3: Client-Side WebSocket Connection

Use JavaScript in the client to connect:

```
const socket = new WebSocket("ws://localhost:5000/ws");
```

```
socket.onopen = function () {
    console.log("Connected");
    socket.send("Hello Server");
};
```

```
socket.onmessage = function (event) {
    console.log("Message from server:", event.data);
};
```

```
socket.onclose = function () {
    console.log("Disconnected");
};
```

```
};
```

Summary

- WebSocket allows real-time, bidirectional communication.
- Ideal for chat, live dashboards, gaming, etc.
- Implementation involves enabling WebSocket middleware, handling socket connections, and managing messages.
- Benefits include reduced latency, minimal overhead, and persistent connections.

This explanation is based on typical WebSocket use cases and ASP.NET Core WebSocket middleware documentation.